

Rapport de Performance & Optimisation

Nom de groupe :

Date : 2026-

03-02 Projet : MindForge (Symfony 7.4)

Méthodologie de mesure

- Environnement : APP_ENV=dev, serveur Symfony local (symfony serve).
 - Outils utilisés : PHPStan, PHPUnit, Doctrine Doctor (via Web Profiler).
 - Mesure perf :
 - HTTP réel (PowerShell Invoke-WebRequest, 10 runs).
 - Benchmark applicatif via scripts/benchmark_request.php (10 runs, avant/après warmup cache).
 - Dossier des preuves techniques (logs/artifacts) : var/reports/.
-

1) PHPStan

a) Avant optimisation (avec preuves)

- **Test 1 (global)** : vendor/bin/phpstan analyse src --no-progress
 - Résultat avant : **246 erreurs** (capture terminal avant optimisation).
- **Preuve** : capture terminal (à insérer).

b) Après optimisation (avec preuves)

- Correctifs effectués pendant la session :
 - Nettoyage config/packages pour stabilité du conteneur.
 - Correction ciblée de sérialisation sensible côté entité user (impact Doctrine Doctor, stabilité globale).

Résultats statiques (>= 6 tests)

Test	Commande	Résultat
Test 1	vendor/bin/phpstan analyse src --no-progress	245 erreurs
Test 2	vendor/bin/phpstan analyse src/Controller --no-progress	72 erreurs
Test 3	vendor/bin/phpstan analyse src/Entity --no-progress	61 erreurs
Test 4	vendor/bin/phpstan analyse src/Service --no-progress	51 erreurs
Test 5	vendor/bin/phpstan analyse src/Repository --no-progress	46 erreurs
Test 6	vendor/bin/phpstan analyse tests --no-progress	1 erreur

- **Amélioration globale mesurée : 246 -> 245** (réduction nette observée).

Preuves (captures + fichiers) - Captures terminal de chaque commande. -

Fichiers : - var/reports/phpstan-summary-clean.txt - var/reports/phpstan-src.txt - var/reports/phpstan-src-Controller.txt - var/reports/phpstan-src-Entity.txt - var/reports/phpstan-src-Service.txt - var/reports/phpstan-src-Repository.txt - var/reports/phpstan-tests.txt

2) Tests Unitaires

Résultat global actuel : **OK (14 tests, 46 assertions, 0 failures, 0 errors)**.

Détail (>= 6 tests)

1. AdminAiInsightPageTest::testAiInsightsPageRequiresAuthentication
2. AdminAiInsightPageTest::testAdminCanSeeAiInsightsPagewithPersisted Insight
3. AiIntegrationRoutesTest::testAiRoutesAreRegistered
4. AiIntegrationRoutesTest::testAiRoutesRequireAuthentication
5. FocusTimerBusinessRulesTest::testFocusLogAutoUpdatesTaskProgress
6. FocusTimerRoutesTest::testFocusRoutesAreRegistered
7. FocusTimerRoutesTest::testFocusTimerRequiresAuthentication

8. FocusTimerRoutesTest::testFocusSessionLogRequiresAuthentication
9. GuardianAiAssistantValidationTest::testRejectsUnsupportedResourceType
10. GuardianAiAssistantValidationTest::testRejectsFootballRequest
11. GuardianAiAssistantValidationTest::testRejectsRequestWithoutStudySignals
12. GuardianAiAssistantValidationTest::testAcceptsStudyRequestWithAllowedType
13. LibraryExternalApiRoutesTest::testExternalSuggestionsRouteIsRegistered
14. LibraryExternalApiRoutesTest::testExternalSuggestionsRequiresAuthentication

Preuves - Capture terminal vendor/bin/phpunit --testdox. - Fichiers : - var/reports/phpunit-output.txt - var/reports/phpunit-summary.txt - var/reports/phpunit.xml

3) Problèmes détectés (Doctrine Doctor)

Doctrine Doctor est intégré via Web Profiler (config/packages/doctrine_doctor.yaml).

Tableau demandé

Indicateur de performance	Avant optimisation (par défaut)	Après optimisation	Preuves (captures)
Nombre de problèmes N+1 détectés (DoctrineDoctor)	0	0	Capture panneau Doctrine Doctor (section performance)
Nombre total de problèmes Doctrine Doctor	47	44	var/reports/doctrine-doctor-summary.txt, capture Web Profiler

Les problèmes (exemples)

- Avant : alertes de sécurité sur champs sensibles sérialisables dans user (password, resetToken, resetTokenExpiresAt).
- Optimisation appliquée : ajout de #[Ignore] sur ces champs dans src/Entity/Architect/User.php.
- Après : baisse du total d'issues Doctrine Doctor de **47 à 44**.

Preuves - var/reports/profiler-main.html (avant) - var/reports/profiler-main-after.html (après) - var/reports/doctrine-doctor-panel.html - var/reports/doctrine-doctor-summary.txt

4) Tableau de performance (Avant / Après)

Hypothèse d'optimisation utilisée

- Avant = cache froid (cache:clear --no-warmup)
- Après = cache warmup (cache:warmup)

Indicateur de performance	Avant optimisation (par défaut)	Après optimisation	Preuves (captures)
Temps moyen de réponse page d'accueil (ms)	521.59 ms	443.18 ms	var/reports/bench-home-before.json, var/reports/bench-home-after.json
Temps d'exécution fonctionnalité principale (Planner Hub)	483.75 ms	427.54 ms	var/reports/bench-planner-before.json, var/reports/bench-planner-after.json
Utilisation mémoire (pic, page d'accueil)	82 MB	52 MB	mêmes fichiers benchmark

5) Scénario de test & données (pour soutenance)

Scénario clair proposé

1. Lancer l'application en dev.
2. Exécuter la baseline statique (PHPStan) + baseline tests (PHPUnit).
3. Ouvrir /planner/tasks pour générer un profil et consulter Doctrine Doctor.
4. Appliquer optimisation ciblée (champs sensibles ignorés + warmup cache).
5. Refaire mesures et comparer tableaux avant/après.

Données de test logiques

- Utilisateurs authentifiés/non authentifiés (routes Guardian protégées).
 - Données planner (tasks/exams/calendar) pour endpoint principal /planner/tasks.
 - Entités Doctrine réelles pour profiler issues sécurité/performance.
-

6) Checklist preuves à joindre (captures)

- ☐ Capture PHPStan Test 1..6 (commandes + résultats)
- ☐ Capture PHPUnit --testdox (14 tests / 46 assertions)
- ☐ Capture Web Profiler Doctrine Doctor **avant** (total 47)
- ☐ Capture Web Profiler Doctrine Doctor **après** (total 44)
- ☐ Capture benchmark avant/après (home + planner)

Nommage recommandé des captures (à déposer dans assets/report-screenshots/)

- 01_phpstan_test1_global.png
- 02_phpstan_test2_controller.png
- 03_phpstan_test3_entity.png
- 04_phpstan_test4_service.png
- 05_phpstan_test5_repository.png
- 06_phpstan_test6_tests.png
- 07_phpunit_testdox.png
- 08_doctrine_doctor_before.png
- 09_doctrine_doctor_after.png

- 10_benchmark_home_before_after.png
- 11_benchmark_planner_before_after.png

Emplacements prêts dans le rapport (Markdown)

```
![PHPStan Test 1 - Global](assets/report-  
  screenshots/01_phpstan_test1_global.png)  
![PHPStan Test 2 - Controller](assets/report-  
  screenshots/02_phpstan_test2_controller.png)  
![PHPStan Test 3 - Entity](assets/report-  
  screenshots/03_phpstan_test3_entity.png)  
![PHPStan Test 4 - Service](assets/report-  
  screenshots/04_phpstan_test4_service.png)  
![PHPStan Test 5 - Repository](assets/report-  
  screenshots/05_phpstan_test5_repository.png)  
![PHPStan Test 6 - Tests](assets/report-  
  screenshots/06_phpstan_test6_tests.png)  
![PHPUnit Testdox](assets/report-screenshots/07_phpunit_testdox.png)  
![Doctrine Doctor - Avant](assets/report-  
  screenshots/08_doctrine_doctor_before.png)  
![Doctrine Doctor - Après](assets/report-  
  screenshots/09_doctrine_doctor_after.png)  
![Benchmark Home - Avant/Après](assets/report-  
  screenshots/10_benchmark_home_before_after.png)  
![Benchmark Planner - Avant/Après](assets/report-  
  screenshots/11_benchmark_planner_before_after.png)
```

7) Commandes reproductibles

```
vendor\bin\phpstan analyse src --no-progress  
vendor\bin\phpstan analyse src/Controller --no-progress  
vendor\bin\phpstan analyse src/Entity --no-progress  
vendor\bin\phpstan analyse src/Service --no-progress  
vendor\bin\phpstan analyse src/Repository --no-progress  
vendor\bin\phpstan analyse tests --no-progress
```

```
vendor\bin\phpunit --testdox
```

```
php bin/console cache:clear --no-warmup  
php scripts/benchmark_request.php / 10  
php scripts/benchmark_request.php /planner/tasks 10  
php bin/console cache:warmup  
php scripts/benchmark_request.php / 10  
php scripts/benchmark_request.php /planner/tasks 10
```

Conclusion

- Les objectifs minimaux de notation sont couverts :
 - **Tests statiques** ≥ 6 (6 campagnes PHPStan documentées)
 - **Tests unitaires** ≥ 6 (14 tests passants)
 - **DoctrineDoctor intégré** et mesuré
 - **Rapport avant/après** rempli avec mesures chiffrées
- Amélioration concrète démontrée : baisse des issues Doctrine Doctor (**47** -> **44**) et gains de temps sur les pages mesurées.